```
export const fetchData = (): VenueThunk => {
  return async (dispatch, getState) => {
    const response = await fetch(
      `/tickets.json?concert_id=${getState().concertId}`
    )
    const tickets = await response.json()
    dispatch({ type: "setTickets", tickets })
  }
}
```

It's the exact same logic, but instead of an inner function inside the useEffect hook, it's an inner function inside the thunk. We're declaring the inner function as async and using await on the fetch call, which means the client-side state will only change after the server call is complete.

One other thing to mention is the typing on the return value of the outer function, VenueThunk. We use TypeScript's type command to define VenueThunk as an alias for a more complicated generic type based on Redux Thunk's existing ThunkAction type. The arguments to the generic include the following:

- The expected return type of the inner function. For us it's void, but sometimes these actions can return JavaScript promises if you want more asynchronous behavior.
- The type of the state of the reducer, which for us is VenueState
- The type of an optional argument that you can get Redux Thunk to add to each call. We're not doing that, so the argument is null.
- A type that encapsulates all the actions that might be dispatched by the inner function. We're just using the VenueAction here, so we aren't limiting ourselves. If we were being super careful, we might want to use a specific action or union of actions.

With that in place, all the asynchronous code can be moved out of the App component, leaving it much shorter:

chapter_12/02/app/javascript/components/app.tsx